1. **PYNQ SoC framework is used to command and control the SoC and RTL LENET Neural Network within the synthesized bitstream.**

```
[1]: from pynq import Overlay
     ol = Overlay("/home/xilinx/jupyter_notebooks/LENET/lenet.bit")
     print(list(ol.ip_dict))   # should list 'top_0', 'axi_gpio_0', etc.

     ['axi_gpio_0', 'top_0', 'processing_system7_0']

[2]: print(hex(ol.ip_dict['top_0']['phys_addr']))       # 0x40000000
     print(hex(ol.ip_dict['axi_gpio_0']['phys_addr']))  # 0x40010000

     0x40010000
     ,0x40000000

[3]: base = ol.ip_dict["axi_gpio_0"]["phys_addr"]

[4]: from pynq import Overlay, MMIO
     mm = MMIO(base, 0x10000)
     mm.write(0x00, 0x3)
```

2. **PYNQ Framework allows for read and write of hardware specific registers and DDR memory to load test frames for neural network.**

```
# ---------- 3) AXI-Lite control health: read ap_ctrl, decod
def decode_ap_ctrl(val):
    return {
        "ap_start":      (val >> 0) & 1,
        "ap_done":       (val >> 1) & 1,
        "ap_idle":       (val >> 2) & 1,
        "ap_ready":      (val >> 3) & 1,
        "auto_restart":  (val >> 7) & 1,
    }
ip = ol.top_0
```

```
# If .hwh is present, show the friendly reg map
try:
    print("\n---- top_0 register map ----")
    print(ip.register_map)
    has_hwh = True
except Exception:
    print("\n(top_0 .hwh not parsed; using raw offsets)")
    has_hwh = False
```

```
,---- top_0 register map ----
,RegisterMap {
,  CTRL = Register(AP_START=0, AP_DONE=0, AP_IDLE=1, AP_READY=0, RESE
_3=0),
,  GIER = Register(Enable=0, RESERVED=0),
,  IP_IER = Register(CHAN0_INT_EN=0, CHAN1_INT_EN=0, RESERVED_0=0),
,  IP_ISR = Register(CHAN0_INT_ST=0, CHAN1_INT_ST=0, RESERVED_0=0),
,  image_r_1 = Register(image_r=write-only),
,  image_r_2 = Register(image_r=write-only),
,  conv1_out_1 = Register(conv1_out=write-only),
,  conv1_out_2 = Register(conv1_out=write-only),
,  pool1_out_1 = Register(pool1_out=write-only),
,  pool1_out_2 = Register(pool1_out=write-only),
,  conv2_out_1 = Register(conv2_out=write-only),
,  conv2_out_2 = Register(conv2_out=write-only),
,  pool2_out_1 = Register(pool2_out=write-only),
,  pool2_out_2 = Register(pool2_out=write-only),
,  flat_out_1 = Register(flat_out=write-only),
,  flat_out_2 = Register(flat_out=write-only),
,  fc1_out_1 = Register(fc1_out=write-only),
,  fc1_out_2 = Register(fc1_out=write-only),
,  fc2_out_1 = Register(fc2_out=write-only),
,  fc2_out_2 = Register(fc2_out=write-only),
,  prediction_reg = Register(prediction_reg=0),
```

3. **The desired test image is loaded into the input registers and read back from DDR to ensure the correct image is loaded into the hardware.**

```
# --- config ---
IMG_TXT_PATH = "/home/xilinx/jupyter_notebooks/LENET/imgs/img_5.txt"
H, W = 28, 28  # MNIST size
DTYPE = np.float32   # change to np.uint8 if your txt is 0..255 integers
```

```
# Load values from text file and reshape
img_vals = np.loadtxt(IMG_TXT_PATH, dtype=DTYPE).reshape(H * W)
```

```
# Allocate DDR-backed buffer
img_buf = allocate(shape=(H * W,), dtype=DTYPE)
img_buf[:] = img_vals[:]
img_buf.flush()  # flush to DDR (good before PL reads)
```

```
# Read back & reshape for display
img_back = np.array(img_buf).reshape(H, W)
```

Image from DDR @ 0x15849000

DDR physical address: 0x15849000

4. **Each layer is prepared for arithmetic checkout of the neural network convolution and the final layer calculates the prediction of the digit.**

```
# Allocate DDR-backed buffers
conv1_out = allocate((4704,), dtype=np.float32)
pool1_out = allocate((1176,), dtype=np.float32)
conv2_out = allocate((1600,), dtype=np.float32)
pool2_out = allocate((400,), dtype=np.float32)
flat_out  = allocate((400,), dtype=np.float32)
fc1_out   = allocate((120,), dtype=np.float32)
fc2_out   = allocate((84,), dtype=np.float32)
```

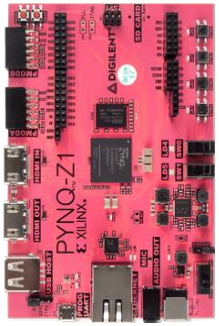
```
# --- Write AXI-Lite control registers ---
mm.write(0x10, img_buf.physical_address)
mm.write(0x1C, conv1_out.physical_address)
mm.write(0x28, pool1_out.physical_address)
mm.write(0x34, conv2_out.physical_address)
mm.write(0x40, pool2_out.physical_address)
mm.write(0x4C, flat_out.physical_address)
mm.write(0x58, fc1_out.physical_address)
mm.write(0x64, fc2_out.physical_address)
```

```
# Write debug/control registers
mm.write(0x94, 0x1EAF0001)  # magic
mm.write(0x7C, 0x0)         # ctrl
mm.write(0x88, 0x0)         # clear status
```

5. **Finally, the inference ran within the PYNQ framework and polls the prediction register after the neural network finishes processing, yielding the correct prediction, within this test, the input digit is 5, and the prediction is 5.**

```
[39]: # --- Start accelerator ---
      mm.write(0x00, 0x1)  # ap_start = 1
      # --- Poll for completion (bit1 = ap_done) ---
      while (mm.read(0x00) & 0x2) == 0:
          pass
      # --- Read result + stats ---
      prediction = mm.read(0x70)
      status     = mm.read(0x88)
      bytes_rd   = mm.read(0xA0)
      bytes_wr   = mm.read(0xAC)
      cycles     = mm.read(0xB8)

      print(f"Prediction: {prediction}")
      print(f"Status: 0x{status:08X}")
      print(f"Bytes read: {bytes_rd}")
      print(f"Bytes written: {bytes_wr}")
      print(f"Cycle count: {cycles}")
```




```
Prediction: 5
,Status: 0x00000000
,Bytes read: 0
,Bytes written: 1
,Cycle count: 33940
```